

ML Specification Report — playground-series-s3e3

Employee Attrition Prediction · our from-scratch agent (GBDT pool + tree-search v3)

Figures grounded in the competition's `facts.json`, `eda_summary.json`, `STATUS.md`, `experiments_tree_v3.json`, and the study's frozen three-way score table `benchmark_results/three_way_scores.csv`.

Contents

• Employee Attrition Prediction · our from-scratch agent (GBDT pool + tree-search v3)	1
• Overview	1
• Data	1
• Models & Architecture	3
• Training procedures	3
• Inference procedures	4
• Evaluation & Benchmarking	4
• Final-Architecture Re-Run (v5 pipeline, Stage 0.5 in-lane) — 2026-08	5

Overview

The task is to rank 1,119 employees by attrition probability (metric: ROC-AUC). We solve it with a from-scratch pipeline — a pool of gradient-boosted trees (LightGBM / XGBoost / CatBoost) refined by four linear self-improvement rounds and a 22-node tree search that ends on a single heavily-regularized tuned-LGB champion. The tree-search v3 sweep reaches **OOF AUC 0.841442** (node #11, 42 features); the competition's best recorded internal CV is **0.845051**, an 80-node scale-run KITCHENBLEND logged as `facts.json` best (experiment_id 8). In the frozen three-way score table our agent's `mine_local` is **0.845051**, while the NVIDIA reproduce-agent — copying a public ensembling kernel verbatim — records **0.8758**, and `local_winner` is set to **nvidia**. This is a small, noisy 1,677-row dataset, so the honest headline is that the well-tuned public kernel retains a clear edge here.

Why it matters. Predicting which employees will leave lets organizations act before costly turnover — recruitment, onboarding, and lost institutional knowledge — by targeting retention where it pays off.

Data

Purpose of Data. Predict whether an employee leaves the company (attrition) — a binary-classification Playground Series episode derived from the classic IBM HR attrition table. **Data Format** is clean **tabular CSV**, a mix of numeric and categorical columns. **Data Volume** is **1,677 training rows / 1,119 test rows**, with 33 raw features (**25 numeric + 8 categorical**) once `id` and the `Attrition` target are removed.

Data Quality is high: **no missing values** in either split, **0 duplicate rows**, and **no unseen test categories** in any categorical column. Two modelling wrinkles dominate: (a) strong class imbalance — only **11.9% of employees churned** (`Attrition=1`; 200 of 1,677), which forces stratified CV; and (b) three raw columns are **zero-variance constants** — `EmployeeCount` (`=1`), `StandardHours` (`=80`), `Over18` (`"Y"`) — and are dropped. The strongest EDA signals, all consistent with HR intuition, were:

- **OverTime** is the single strongest categorical driver — **22.0% attrition when "Yes" vs 8.8% when "No"**.
- **MaritalStatus** : **Single 19.8% vs Divorced 4.9%** attrition.
- The strongest numeric correlations are all **negative (protective)**: `StockOptionLevel` (Spearman -0.227), `MonthlyIncome` (-0.184), `YearsAtCompany` (-0.170), `Age` (-0.169), `JobLevel` (-0.163), `TotalWorkingYears` (-0.160).
- Heavy internal collinearity: tenure features (`YearsAtCompany` / `YearsInCurrentRole` / `YearsWithCurrManager`) correlate 0.75–0.79, and `JobLevel` $\leftrightarrow$ `MonthlyIncome` $r \approx 0.91$ — a redundancy the tree search later exploits.
- **No single-feature leakage** (no column reaches $AUC > 0.75$ alone). With only 1,677 rows, **per-fold AUC swings 0.79–0.89** — expected variance, not a bug.

Annotation Guidelines. The label is `Attrition` $\in \{0, 1\}$ (1 = left the company). Submissions are **attrition probabilities** scored by AUC — a ranking, not a hard 0/1 decision.

Feature Set. From the 33 raw columns we drop 3 constants and engineer up to **48 features**, grouped below:

Group	Features
Encoded (label + frequency)	7 categoricals — <code>BusinessTravel</code> , <code>Department</code> , <code>EducationField</code> , <code>Gender</code> , <code>JobRole</code> , <code>MaritalStatus</code> , <code>OverTime</code> (each $\rightarrow$ <code>_le</code> + <code>_freq</code>)
Tenure ratios	<code>role_tenure_ratio</code> , <code>mgr_tenure_ratio</code> , <code>promo_ratio</code> , <code>company_tenure_ratio</code>
Income	<code>income_per_joblevel</code> , <code>income_per_year_worked</code>
Satisfaction	<code>satisfaction_avg</code> , <code>satisfaction_min</code>
Interaction / derived	<code>overtime_joblevel</code> , <code>age_at_join</code> , <code>companies_per_year</code>
Dropped (zero variance)	<code>EmployeeCount</code> , <code>StandardHours</code> , <code>Over18</code>

Encoding is **label + frequency**, not WOE/target encoding — our pipeline deliberately avoids the weight-of-evidence route (which belongs to the NVIDIA benchmark; see the Evaluation section). The champion node trims further to **42 features**, dropping the 5 raw tenure columns plus `income_per_year_worked` once their engineered ratios exist.

Splitting strategy. A single canonical **Stratified 5-Fold CV on Attrition** (`shuffle = True`, `seed = 42`), held **fixed across every experiment and every tree-search node** so scores are directly comparable. Given the tiny 1,677-row split, a full 5-fold pass is cheap enough to run inside each Optuna trial (no fold-0 proxy needed). The leaderboard was not used (unattended run — submission files generated but not uploaded), so **all scores here are CV-only OOF AUC**.

Models & Architecture

Purpose of Architecture. A binary attrition classifier that maximizes ROC-AUC. **Architecture Type** is a **gradient-boosted decision-tree pool** — LightGBM as the primary learner, with XGBoost and CatBoost members available for blending. Unusually for this project, the CV **champion is a solo tuned-LGB model** (tree-search node #11), which beat every weighted / rank-average blend the search tried on this small dataset.

Each member takes the same **Input Format** — a numeric feature matrix with categoricals label/frequency-encoded — of **Input Dimension 42 features** for the champion (48 in the full pool), a 1-D vector per row with no spatial/sequence structure.

Architecture Description. Each GBDT member is an additive ensemble of decision trees grown by histogram-based, leaf-wise gradient boosting. The champion is deliberately **shallow and heavily regularized** to fight overfitting on 1,677 rows: LightGBM with `num_leaves = 3`, `max_depth = 4`, `min_child_samples = 60`, `n_estimators` up to 2000 with early stopping, `learning_rate` ≈ 0.0503 . Where blends are used, the layer on top is a convex weighted average (Dirichlet-sampled) or a rank-average of member OOF predictions. **Model Complexity** is best expressed as `trees` $\times$ `leaves` rather than a dense parameter count: with only 3 leaves per tree, each LGB member is an extremely low-capacity learner by design — the search repeatedly confirmed that *regularization over capacity* is what this dataset rewards.

Training procedures

Training proceeded as **four linear self-improvement rounds followed by a tree search**, all on the shared 5-fold split and measured on the same OOF scale:

Stage	Configuration	OOF AUC
1	generic LGB+XGB+CAT blend (no feature engineering)	0.81624
2	+ 48 engineered features, imbalance-weighted 3-model blend	0.819008
3	imbalance weighting removed, lighter/regularized LGB	0.832925
4	Optuna-tuned LGB (50-trial full-CV-AUC objective, solo)	0.837305
5	+ seed-bag, 5-way weighted blend	0.837776
6	CatBoost native categoricals (diagnostic, solo)	0.814259
7	6-way rank-average blend (linear-iteration best)	0.838140
8	tree-search v3 (22 nodes; node #11 = tuned-LGB on 42 feats)	0.841442

The key Reflexion finding at stage 3: because **AUC is a ranking metric**, `scale_pos_weight` / `class_weights` mostly perturbed the loss landscape rather than helping ranking — *removing* all imbalance weighting jumped LGB from 0.819 $\rightarrow$ 0.833. The champion (stage 8, node #11) traces to a comp-local hypothesis — dropping the redundant raw tenure columns once their ratios exist — verified as a real +0.0041 gain over the tuned-LGB root.

The **Loss Function** is binary log-loss (LightGBM `binary` objective) with model selection on AUC. The **Optimization Algorithm** is gradient boosting (histogram, leaf-wise) — **not** SGD/ADAM — with blend

weights found by Dirichlet sampling / weight search. The **Learning Rate** is the boosting shrinkage (champion LGB `learning_rate ≈ 0.0503`; the LGB_orig member used 0.03); there is **no Learning Rate Scheduler** (*N/A — GBDT convergence is governed by CV early-stopping, not a schedule*) and **no Batch Size** (*N/A — full-dataset histogram boosting, not mini-batched*).

Training Duration for the tree-search v3 sweep was **~34 s of wall-clock over 22 model fits** (22 evaluated nodes, 0 failed); the one-time Optuna LGB tune was 50 trials / 111.3 s, and a solo LGB fits in ~1 s. **Training Memory Consumption Limits** were not explicitly capped — a $1,677 \times 48$ float matrix is trivial on the single arm64 CPU machine; peak memory was **not recorded**. **Transfer Learning** is *N/A (no pretrained weights)*; its analogue here is **experience-library prior injection** — `suggest_priors` returned 7 bullets (P0–P6) that shaped the mutation queues (directionally correct: 0 wasted searches down closed dead-ends like re-tuning CatBoost, though the biggest score-mover was a comp-local idea, not a transferred one). **Data Augmentation Techniques** are *N/A (tabular)*; the analogues are feature engineering (the 48 features) and seed-bagging (a seed = 2024 bag, which the weight search zeroed out — little extra diversity on an already-directly-optimized model).

Reproducibility Standards are strict: a fixed fold seed (42) shared across all experiments, logged Optuna and seed-bag seeds, and a faithful-replay contract. Per the benchmark log, a faithful replay of the committed tree reproduces **22/22 nodes exactly ($\Delta = 0$)**.

Inference procedures

Decision Threshold is *N/A* — AUC is a ranking metric, so we submit raw probabilities and never threshold (a 0.5 cut would only be needed if hard labels were required). **Inference Duration** was **not separately profiled**; GBDT scoring of the 1,119 test rows is sub-second per member on CPU. **Inference Memory Consumption Limits** are negligible and **not capped** — tree inference on a ~1k-row test set is lightweight.

Evaluation & Benchmarking

Task for Performance Evaluation. Rank the test employees by attrition probability. **Performance Metrics.** ROC-AUC is the sole competition metric; our internal CV climbs **0.81624 → 0.819008 → 0.832925 → 0.837305 → 0.837776 → 0.838140 → 0.841442** across the eight stages, with the tree-search v3 champion (node #11) at **OOF AUC 0.841442**; the separate 80-node scale run recorded in `facts.json` `best` reaches **0.845051**, and that is the figure the frozen three-way table carries as `mine_local`. The competition leaderboard was not used (unattended run), so every figure here is **CV / OOF, not public-LB**.

Performance Benchmarking. The natural comparator is the **NVIDIA reproduce-agent**, which copies the strongest public kernel verbatim:

Agent	Approach	Local CV AUC	Note
NVIDIA	reproduces <code>chunweishen/ps-s03e03-ensembling</code> (verbatim)	0.8758	<code>local_winner</code>
Our agent	from-scratch GBDT pool + tree-search v3	0.845051	behind

NVIDIA leads here. The relevant caveat concerns **weight-of-evidence (WOE) encoding**, which lives in the **NVIDIA kernel, not our pipeline** (ours used label/frequency encoding throughout). When that kernel is audited for target leakage, its WOE turns out to be **fold-safe** — de-leaking leaves the score **unchanged at 0.8758**, so NVIDIA's win here is **genuine**, not a leakage artifact. On this small, noisy ~1.7k-row S3-era problem the well-optimized public ensembling kernel holds a clear, honest edge over our from-scratch pipeline; the reproduce-vs-originate gap narrows only on the larger, newer seasons analyzed separately.

Final-Architecture Re-Run (v5 pipeline, Stage 0.5 in-lane) — 2026-08

All 20 baseline competitions were re-run in an isolated root (`~/benchruns/myagent-rerun`) under one frozen architecture (`docs/rerun_manifest.json`): every lane runs the **full v5 pipeline including the Stage 0.5 problem dossier** (dossier.json written in-lane before modeling), holds no credentials, and is checked by a per-lane transcript audit. Leaderboard scores for the re-run were produced by the separate credentialed operator scoring step and are recorded in `benchmark_results/rerun_three_way.csv` ; the figures below revise nothing in the sections above.

- **Lane**: done 2026-08-14, 20 min.
- re-run local CV: **ROC-AUC 0.841605** pooled 5-fold OOF, equal-weight blend of four seed-bagged members (no fitted weights, no optimism correction needed).
- Public / private leaderboard for this re-run (`benchmark_results/rerun_three_way.csv`): **Public 0.89449 / Private 0.87157**; `winner_priv` = **nvidia** (NVIDIA 0.89537, AIDE 0.86863).